

Cheatsheet de Numpy y Pandas

Guia de repaso rápido



1. NumPy — Operaciones fundamentales (import numpy as np)

- **Creación y dimensiones:**

- `arr=np.array([1, 2, 3])`, `np.array([[1, 2, 3],[4, 5, 6]])`, `np.zeros((2, 3))`: Crea un array n-dimensional.
- `np.linspace(0, 10, 100)`: Genera 100 valores equiespaciados entre 0 y 10 (ideal para ejes X).
- `arr.shape` / `arr.reshape(filas, cols)`: Consulta / modifica la dimensión del array.

- **Datos sintéticos (testing de gráficos):**

- `np.random.randn(1000)`: Genera 1000 valores con distribución normal (para histogramas / KDE).
- `np.random.randint(0, 100, 50)`: 50 números enteros aleatorios entre 0 y 100.

- **Estadística rápida:**

- `np.mean(arr)`, `np.median(arr)`, `np.std(arr)`: Media, mediana y desviación estándar.

2. Pandas — Carga y exploración inicial (import pandas as pd)

- **Carga de datos:**

- **df = pd.read_csv('datos.csv')**: Carga un dataset estructurado en un DataFrame.

- **Inspección y dimensiones:**

- **df.shape**: Devuelve las dimensiones exactas del DataFrame en formato (filas, columnas).
- **df.head()** / **df.tail()**: Muestra las 5 primeras o últimas filas del conjunto de datos.
- **df.info()**: Resumen de columnas, tipos de datos y recuento de valores nulos.
- **df.describe()**: Resumen estadístico rápido (media, mín, máx, cuartiles) de variables numéricas.

3. Pandas — Limpieza de datos (nulos y duplicados)

- **Gestión de valores nulos (NaN):**

- **df.dropna():** Elimina cualquier fila que contenga valores nulos (NaN).
- **df.fillna(valor):** Imputa los valores nulos con un número o estadística específica (ej. la mediana).
- **df.ffill(axis=0) / df.bfill(axis=0):** Forward/Backward fill (axis=0 por filas, axis=1 por columnas).

Rellena el nulo arrastrando el valor de la fila anterior (ffill) o posterior (bfill). Clásico en series temporales.

- **df.interpolate(axis=0, method='polynomial'):** Rellena los huecos calculando matemáticamente la línea recta / curva entre el dato anterior y el posterior. Muy estético para gráficos de líneas.

- **Calidad de registros:**

- **df.drop_duplicates():** Borra filas duplicadas idénticas para evitar sesgos al graficar.
- **df['col'].replace({'España ': 'España', 'USA': 'EEUU'}):** Reemplaza valores exactos. Útil para fusionar categorías que significan lo mismo antes de un gráfico.
- **df.rename(columns={'col_fea': 'Ventas (€)}):** Renombra columnas. Tip: Hazlo antes de graficar y Matplotlib/Seaborn usarán este nombre bonito para la etiqueta del eje automáticamente.

3. Pandas — Limpieza de datos (nulos y duplicados)

- **Corrección de Tipos de Datos (evita ejes desordenados):**

- **`pd.to_datetime(df['fecha'])`:** Convierte texto plano a formato cronológico real. Obligatorio para que los gráficos de series temporales fluyan en orden.
- **`pd.to_numeric(df['col'], errors='coerce')`:** Fuerza a convertir una columna a números. Si choca con un texto ("N/A", "10€"), lo transforma de forma segura en NaN sin romper el código.
- **`df['col'].astype('category')`:** Convierte texto en variables categóricas, permitiendo establecer un orden personalizado (ej. Bajo < Medio < Alto). # `category = pd.CategoricalDtype(categories=['Bajo', 'Medio', 'Alto'], ordered=True)`

- **Limpieza de Texto / Strings (evita categorías duplicadas):**

- **`df['col'].str.strip()`:** Elimina los espacios invisibles al principio o final (hace que "Gato " y "Gato" se agrupen correctamente).
- **`df['col'].str.lower()` / `df['col'].str.upper()`:** Convierte el texto a minúsculas o mayúsculas. Ideal para evitar que "Madrid" y "MADRID" salgan como dos barras distintas en el gráfico.

4. Pandas — Filtrado, selección y conteo categórico

- Selección de columnas:

- `df[['col1', 'col2']]`: Selecciona columnas específicas por su nombre exacto.
- `df.filter(items=['col1', 'col2'], axis=1)`, `df.filter(like='texto', axis=1)`: Selecciona columnas por coincidencia o regex `df.filter(regex='patron')`.

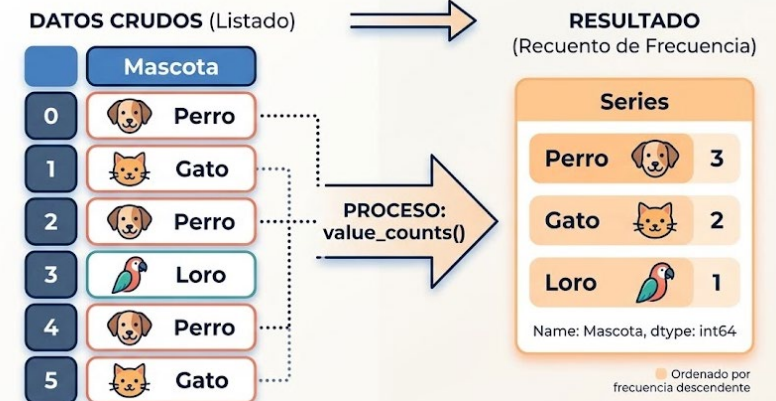
- Indexación y filtrado:

- `df.loc[df['edad'] > 30, :]`, `df.loc['Carlos', :]`: Filtrado lógico / por etiqueta (nombre de fila). Selecciona las filas que cumplen una condición.
- `df.iloc[0:5, 1:3]`: Filtrado por índice posicional. Extrae datos basándose estrictamente en su posición.

- Agregación categórica:

- `df['categoria'].value_counts()`: Frecuencia absoluta por categoría (ideal para gráficos de barras).

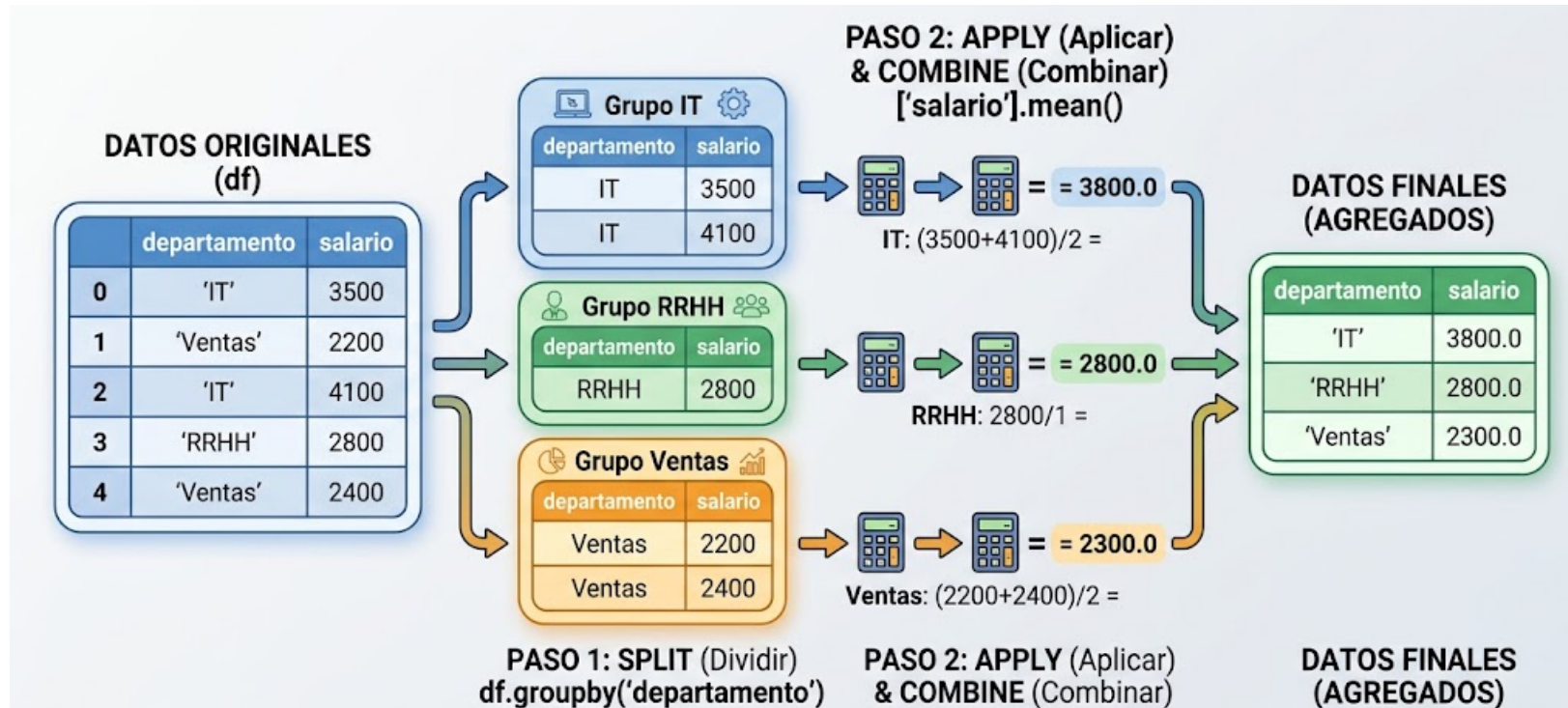
EJEMPLO SENCILLO: `df['Mascota'].value_counts()`



5. Pandas — Agrupación y cruces de tablas

- **Agrupación comparativa (Split-Apply-Combine):**

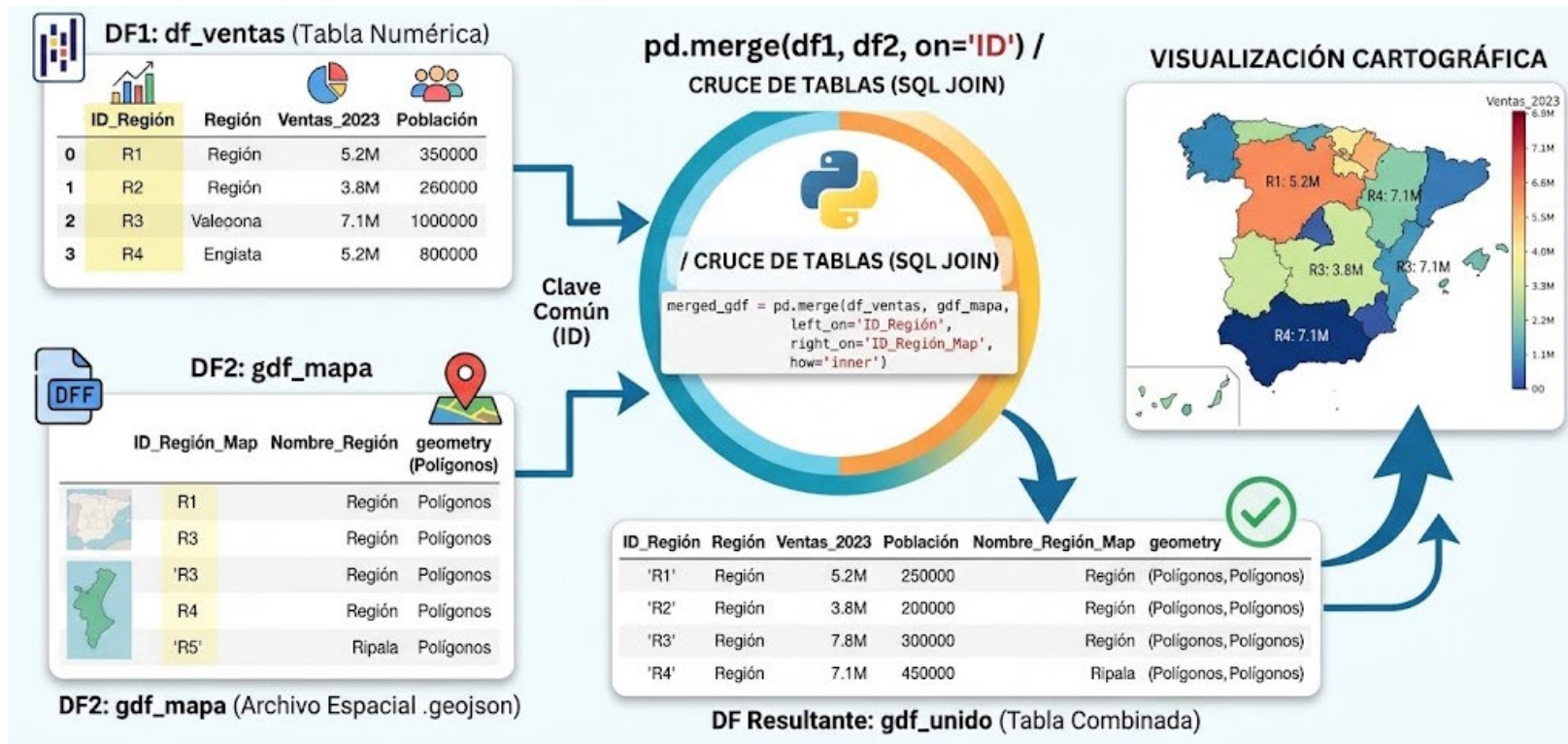
- `df.groupby('departamento')['salario'].mean()`: Agrupa por categoría y calcula la media de otra.
- **Uso en visualización:** Prepara tablas agregadas para comparar grupos en gráficos de barras o líneas.



5. Pandas — Agrupación y cruces de tablas

- **Cruce de tablas (SQL JOIN):**

- `pd.merge(df1, df2, on='ID')`: Cruza dos tablas diferentes usando una columna común como llave.
- **Clave en cartografía**: Permite unir tablas numéricas con archivos espaciales (.geojson) antes de crear mapas.



Cheatsheet de Numpy y Pandas

Guia de repaso rápido
